Sudoku AI Using First-Order Predicate Calculus

Project overview and objectives

Project idea

Represent Sudoku rules as logical constraints and use the Z3 SMT solver to find a satisfying assignment.

- Model each possible digit placement with a Boolean predicate.
- Encode cell, row, column and box rules.
- Preserve the original puzzle clues.

Application goals

- Interactive 9×9 Sudoku board
- Check entries and show conflicts
- Request a hint or full AI solution
- Clear entries while retaining clues

Prerequisites and tools

Concepts

- Boolean connectives and predicates
- Finite domains and constraints
- Satisfiability and logical models

Technology

- Python 3 — solver logic and game state
- Z3 — constraint solving
- Google Colab — notebook runtime
- HTML/CSS/JavaScript — game board
- Colab callbacks — UI/Python connection

Install dependency: `pip install z3-solver`

Problem statement: Sudoku

A standard Sudoku is a 9×9 grid. Fill each empty cell with a digit from 1 to 9 while obeying all rules.

1. Every row contains digits 1–9 exactly once.
2. Every column contains digits 1–9 exactly once.
3. Every 3×3 box contains digits 1–9 exactly once.
4. Given clues remain unchanged.

5	3			7				
6			1	9				
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			
				8			7	9

Predicate representation

Predicate definition

$$\mathrm{Val}(r, c, d) \equiv \text{ cell at row } r\text{, column } c \text{ contains digit } d$$

The notebook represents the predicate with Boolean variables $\texttt{Val}_{r,c,d}$, where $r, c, d \in \{1, \ldots, 9\}$.

- $9 \times 9 \times 9 = 729$ Boolean variables represent candidate placements.
- True means the candidate digit is selected for that cell.
- Z3's model is decoded into a regular 9×9 Python grid.

Logical constraints

1. Exactly one digit per cell

At least one: $\bigvee_{d=1}^{9} \mathrm{Val}(r, c, d)$. For every pair $d_1 < d_2$, forbid both: $\neg\mathrm{Val}(r, c, d_1) \vee \neg\mathrm{Val}(r, c, d_2)$.

2. Row and column uniqueness

For a fixed digit, pairwise constraints prevent it from appearing in two different cells in the same row or column.

3. Box uniqueness and clues

The same pairwise exclusion applies inside each 3×3 box. Every non-zero input clue is asserted as a fixed fact.

Z3 solver implementation

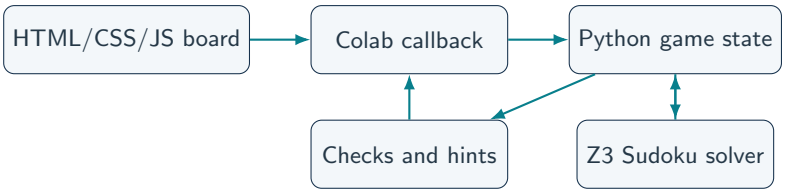
```
Val = {
    (r, c, d): Bool(f"Val_{r}_{c}_{d}")
    for r in DIGITS
    for c in DIGITS
    for d in DIGITS
}
```

- Add Boolean constraints for cells, rows, columns, boxes and clues.
- Call `solver.check()` to test satisfiability.
- If the result is `sat`, read `solver.model()` and decode the selected digit in every cell.

Solver result

`sat` means a model exists; `unsat` means no assignment satisfies the encoded constraints.

Application architecture and interface



Controls

- New Game (two puzzles)
- Number pad and erase
- Check and Clear entries

AI features

- Hint fills a correct value
- Solve with AI shows solution
- Conflicts are highlighted

Validation and results

Verification checks

The notebook's `verify_solution(puzzle, solution)` checks:

1. Each row contains 1–9 exactly once.
2. Each column contains 1–9 exactly once.
3. Each 3×3 box contains 1–9 exactly once.
4. All original non-zero clues are preserved.

Demonstrated outcomes

- Z3 generates a complete assignment when constraints are satisfiable.
- The application supports two puzzles and interactive input.
- Checking, hints, AI solving and clearing are provided.

Test evidence

An assertion verifies the first included puzzle; the notebook prints a success message when the check passes.

Thank you!

Questions and discussion

5	3			7				
6			1	9				
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			
				8			7	9